FRED TALKS

3rd November 2026

CONTENTS

Automating eval design and hillclimbing with Claude
LANCE MARTIN · 2792 WORDS

Agentic Monitoring
BEN CONGDON · 1924 WORDS

What requirements don't you have?
SEANGOEDECKE.COM RSS FEED · 945 WORDS

Automating eval design and hillclimbing with Claude

LANCE MARTIN · 01 OCT 2026 · SOURCE

Principles for designing evals and hillclimbing against them without fooling yourself, and how the claude-api skill's build-eval and hillclimb commands put them to work.

Evaluations provide a signal on how your app or skill is performing on specific tasks. But designing evaluations, and improving performance on them without fooling yourself, is hard. We've added guidance for both to the claude-api skill.

With the skill, you can run `/claude-api build-eval` to build an evaluation inside your codebase, and run `/claude-api hillclimb` to improve your application against it, one change at a time, with a held-out set of examples to catch overfitting.

In this article, we highlight the principles of good eval design and hillclimbing first, then show how Claude Code with the `claude-api` skill applies those principles. We'll close by showing a few examples of these commands.

EVAL DESIGN

Well designed evaluations have a few common elements (Figure 1):

1. **Eval tasks mirror production.** Sample tasks that you care about in "production," or the setting in which the capability or application you are testing will be used. Sometimes tasks are picked because they are easy to generate or they are easy to grade. But it's important to ensure that the task distribution represents what you *actually* care about.

2. **Performance improves with stronger models and more thinking**. More capable models and higher effort levels typically should perform better on an evaluation. If they don't, ambiguous tasks or a miscalibrated grader often are hobbling performance.

3. **There is "passable" headroom at the frontier**. The most capable model at the highest effort should be well below 100% on the evaluation, otherwise you can't reliably judge how changes impact performance. Importantly, the gap should not be explained by impossible or ambiguous tasks: a common tell is that a task fails every evaluation run, regardless of the number of replicates. A good task is one where two domain experts would reach the same verdict and everything the grader checks is stated in the task.

4. **Low run-to-run variance**. High variance is often due to poorly designed, ambiguous tasks or a grader that produces different verdicts on identical output. Variance can also hide in the configuration. For example, effort may not be applied consistently. Also, the environment can affect the results of the evaluation: leftover state from an earlier trial (a file, a git history) can hand the agent the answer.

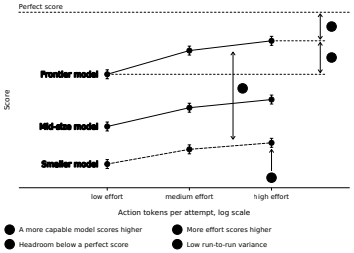
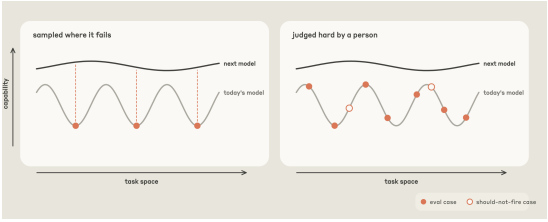


FIG 1 The four elements of a good eval

Adversarial sampling

Model capability is jagged. If you pick cases because today's model fails them, you are sampling the valleys of one model's capability surface (Figure 2). The evaluation can end up measuring that model's failure fingerprint rather than what is intrinsically hard or valuable for your application to do.



Two panels plotting capability across task space, each with today’s model as a jagged curve and the next model as a smoother curve above it. On the left, cases sampled where today’s model fails sit only in its valleys; on the right, cases a person judged hard are spread across peaks and valleys, with a few should-not-fire cases.

Pick hard cases because a human judged them hard: a useful test is to be able to say why a task is hard before you include it. Include cases that are specific failures in your application derived from production traffic, bug reports, or tickets. However, don’t blindly trust user traffic: users sometimes try what they expect to work, so a task distribution drawn strictly from user traffic may skew easy.

/CLAUDE-API BUILD-EVAL

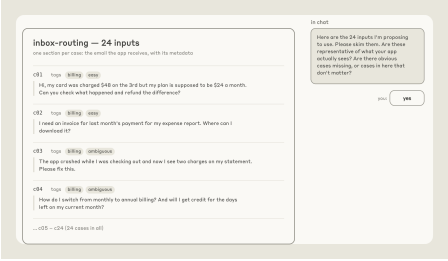
The `build-eval` command in the claude-api skill turns these principles into a guided workflow. When you run `/claude-api build-eval` in Claude Code, Claude interviews you, builds the eval inside your codebase, and pauses for approval at specific points.

Designing examples

Claude helps you sample inputs to build evaluations in this order:

1. Production transcripts, after asking about retention and sensitive data.
2. Bug reports and support tickets.
3. Five to ten cases you write by hand.
4. Cases synthesized from your codebase.

The skill prioritizes production traffic, but it can also generate synthetic data anchored in a few real examples that you provide. The skill instructs Claude to generate a simple page that shows you every input and waits until you confirm them. As an illustration, below we show an example set of inputs for an e-mail router application that the skill may ask the user to review (Figure 3).



The skill's review page for an inbox-routing eval with 24 inputs, listing each case's email text with tags such as billing, easy and ambiguous. Beside it, Claude asks in chat whether the inputs are representative, and the user answers yes.

Validating the grader

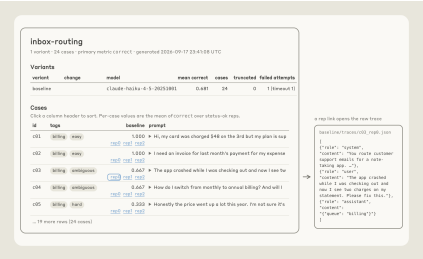
After the inputs, Claude proposes the cheapest grader that fits your application's output:

- **Programmatic verification**: If the output possibilities are constrained, it uses a code based check (exact match, a label from a fixed set, JSON that matches a schema, tests that pass).

- **LLM-as-judge**: It will default to this type of check if the output space is open-ended, with many valid answers but clear quality criteria. In this case, a second model reads the input, the output and a rubric written as checkable claims (not a 1-to-5 scale), and returns a score with its reasoning. If you have a baseline to compare against, the judge instead reads both inputs in random order, without being told which is the baseline, and picks the better one. You pick the judge model, and it should not be the model you are testing.

Claude grades a handful of cases and asks whether you would have scored any of them differently (Figure 4). In general, it is important to read a sample of scored transcripts before believing your evaluator; scoring failures are among the most common ways an evaluation is misconfigured.

When you've validated the grader, the skill tells you the size of the evaluation set (cases × repeats × model, and roughly how long it will take), runs the baseline, and prints the score with a confidence interval. What you get back: the cases, the grader, the runner, one JSON line and one full transcript per case, and a plain page that lists each case's score with a link to its transcript. If you want more than that page shows (e.g., a chart), just ask and Claude will build it as an extra page next to it. By default, these extra pages are static files that open locally and load nothing from the network.



The skill's results page for the inbox-routing eval: a baseline scoring 0.681 mean correct across 24 cases, then a table of per-case scores with a link to each repetition. A rep link opens that case's raw JSON trace, shown alongside.

Diagnostic checks

During the baseline runs mentioned above, Claude checks a number of things:

- **Grader**: Claude runs the grader twice on the same output, and reports whether the verdict changed.
- **Plumbing**: Claude checks for timeouts, API errors, and cut-off answers to ensure infrastructure noise doesn't pass as model variance.
- **Headroom**: if the baseline already scores about 95% or higher, the skill warns the user and alerts that the hillclimb should aim to explore cost or latency rather than quality.

HILLCLIMBING

Now that you have a reliable means of grading your application's performance on a task, you can try to improve it. Hillclimbing is an effective way to tune parameters like effort or prompts, which trade-off cost and performance. Some general tips for choosing where to apply it:

- **Cheap iteration** - It should be inexpensive (in terms of time, cost, and effort) to modify whatever surface you are focused on for hillclimbing. Many internal efforts and customers have focused hillclimbing on text, such as prompts and skills. These are easy to change and revert. In contrast, open-ended modifications to an agent harness during hillclimbing may involve extensive code changes.
- **Attributable** - Changes in the score on your evaluation should be attributable to the surface you are modifying during hillclimbing. For example, several successful applications of hillclimbing have focused on skill triggering. The evaluation metric (the trigger rate for the skill) is directly coupled to the skill description that is being modified.
- **Well-scoped objective** - One common failure mode is an open-ended request to improve performance without careful consideration of the headroom available in the evaluation; an evaluation that's near saturation or a poorly scoped surface (e.g., an open-ended request to update the harness) is more likely to stall. One generally strong objective across various efforts is cost: even if an evaluation is saturated, you can ask Claude to find ways to reduce cost while keeping performance at parity.

Overfitting

Even a well-designed evaluation rarely matches the exact task distribution you care about in production. As a result, "overfitting" to an evaluation is a common problem and results in a system that performs better on an evaluation than on production traffic.

There are many ways an evaluation can "leak" into your harness (the code around the model, including prompts, tools, and loop that calls Claude). For example, consider an evaluation task that benefits from OCR, but OCR is rarely beneficial in your production tasks. The evaluation harness might add an OCR tool to your application, which improves on the benchmark without any impact on production. More broadly, hillclimbing may add features to that harness that address edge cases in the particular evaluation examples you've chosen. These harness additions improve your evaluation score, but don't translate to improvements in production (Figure 5).

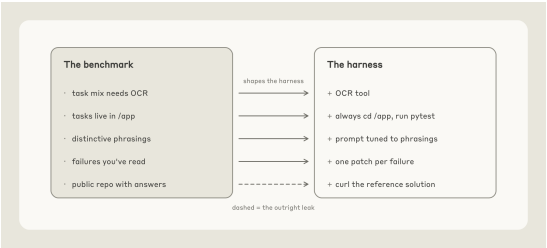


FIG 5 Common causes of harness overfitting.

Three things can help address this:

- **Split the cases**. Use a train set that the hillclimber may read and a test set that is never seen. If the train set scores improve while the test set scores stay flat, then that is a common overfitting warning sign.

- **Never paste failures into the prompt**. If the hillclimber reads the failing transcripts, it should never paste the failure content into the prompt.

- **Keep the answers structurally out of the model's reach**. Models can sometimes "reward hack" by directly finding answers to evaluations.

As discussed below, the claude-api skill applies these principles for you.

/CLAUDE-API HILLCLIMB

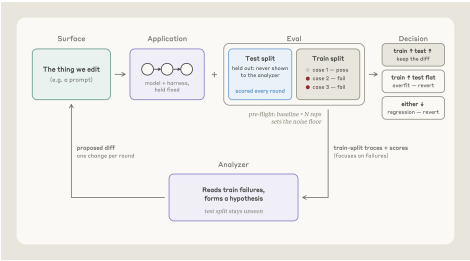
The hillclimb command in the claude-api skill turns these principles into a guided workflow. When you run `/claude-api hillclimb` in Claude Code, Claude iterates to improve against a given evaluation. You choose what changes it can make including:

- Your system prompt
- Skills or instruction files
- Tool descriptions
- Model choice, effort level, and other API parameters
- Your harness code

Before it starts, Claude asks what you want to optimize (e.g., performance, or cost while performance holds) and then splits the evaluation set at random into test and train. With a cost goal, it considers a few common cost drivers, including prompt caching, auditing the prompt for compatibility with the selected model, and picking the model and effort setting.

Before the first round, Claude checks that the eval's noise (how far the score can move by chance alone) is smaller than the smallest improvement you'd act on; if it isn't, it says so and suggests more repetitions or cases.

Each round, Claude reads the previous round’s train transcripts and proposes one change as a patch. It aims each round at a change whose effect can show above the eval's noise: it fixes the failing behavior at its root (e.g., rewrites the section that causes it or adds a missing rule) rather than rewording a line. It then runs evaluation with the patched change. At this point, Claude applies a check: if the `train` set improves but the `test` set is flat, Claude suspects overfitting and reverts the patch. If there is a regression, Claude reverts. If train and test sets improve, it keeps the patch (Figure 6).

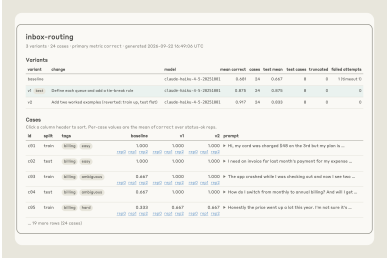


The hillclimbing loop: the thing being edited, such as a prompt, feeds a fixed model and harness that is scored on a held-out test split and a train split. An analyzer reads only the train failures and proposes one diff per round; the diff is kept when train and test both rise, and reverted when only train rises or either score drops.

When the score stalls for two or three rounds, Claude reads each remaining train failure and sorts it by cause. It does the same early if no single fix could gain more than the eval's noise, and suggests more repetitions or cases, rather than spending rounds on changes too small to measure. This step can catch ambiguous evaluation cases, harness errors, or run-to-run variance.

Only legitimate failures are included in more hillclimbing rounds.

When hillclimbing completes, Claude leaves your code at the version that did best on the test set for your goal. It reports the test result against the baseline with confidence intervals (Figure 7). If the gain is within noise, it says so and recommends against merging.



The inbox-routing results page after hillclimbing, comparing three variants on train and test scores. Variant v1, which defines each queue and adds a tie-break rule, is marked best at 0.875 on both; v2, which adds two worked examples, was reverted because train went up while test stayed flat.

EXAMPLES

Hillclimbing for cost reduction

We ran `/claude-api hillclimb` on an internal customer support benchmark with the goal of reducing cost and improving performance. The benchmark included 44 tickets, with 30 used for the search and 14 held out. It started on Opus 4.8 at default (high) effort settings with 74.4% decision accuracy on the search tickets and a token cost of 4.6 cents per ticket.

The hillclimb first audited the prompt, removing mandatory tool-call rituals, a scratchpad step, and contradictory rules. Then it tried Opus 5.5 on low effort. This cleared the baseline accuracy bar at 87.8% and cut cost to 1.9 cents per ticket, less than half the starting cost.

Part of that saving comes from Opus 5.5's pricing: input and output tokens cost 20% less than on Opus 4.8, and cache reads cost 60% less. Because Opus 5.5 cleared the bar, the hillclimb then stepped down a tier to check whether a cheaper model could clear it too. Sonnet 5 on low effort scored about the same, 88.9%, at about half the cost, 1 cent per ticket (Figure 8).

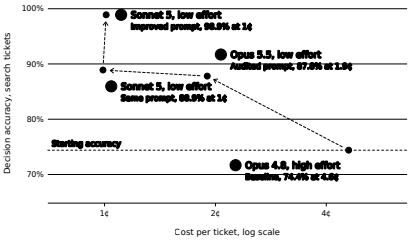


FIG 8 Cost-focused hillclimbing.

Finally, Claude improved the prompt with routing rules and a refund-cap cross-reference, bringing Sonnet 5 to 98.9% at about the same cost. On the 14 held-out tickets that the search never saw, the final configuration scored 90.5% against the original setup's 78.6%, at about one fifth of the cost.

Hillclimbing for performance improvement

Another example is our claude-api skill, which provides guidance on using our APIs and general tips for working with Claude (including the sub-commands discussed in this article). We want to ensure our skill can correctly implement code that uses our APIs, and we built an evaluation set derived from our documentation to test the skill.

On our evaluation, the skill started at 66%. We gave the hillclimber access to documentation and our SDKs, allowing Claude to identify errors and self-correct them (Figure 9). Claude found that the skill was missing coverage of eight features.

Adding sections for them in the skill improved performance to 74%. It then found errors in C# and Java type tables, boosting performance to 77%.

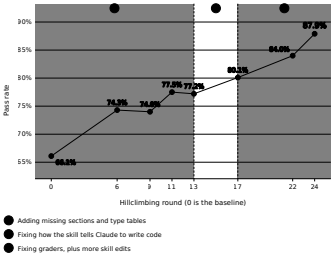


FIG 9 Performance-focused hillclimbing.

After the score stalled for two rounds, Claude analyzed the remaining failures and bucketed them by root-cause. A normal round makes one edit for the most common failure. This step makes no edit; it only sorts every remaining failure by cause. This reflection step was useful in a few ways:

- Reflecting across a collection of failures, the hillclimber found that the skill content was present but Claude was simply writing older API shapes (e.g., from its trained priors). To address, the hillclimber added a table near the top of the skill that guided Claude from the forms it remembered to the current ones: for example, from extended thinking with a fixed token budget, which the API now rejects on recent Opus models, to adaptive thinking, and from older versions of the web search and web fetch tools to the current ones. It also moved the C# and Java warnings against fixed-budget thinking above their adaptive-thinking examples. This improved performance to 80%.

- Tasks that never improved in performance despite addressing obvious content gaps are tells that the example or grader is flawed. One task asked for code that catches one error type, while its grader wanted a chain of at least three. Claude reworded the task. Another grader's instructions contradicted our docs, and testing the real API showed the docs were right. Addressing these, along with more skill edits, brought performance to ~88%.

GETTING STARTED

Note: Run `claude update` first. The claude-api skill ships inside Claude Code, so updating gets you the latest version of these commands.

```
claude update
```

Then, in Claude Code:

```
/claude-api build-eval
/claude-api hillclimb
```

These sub-commands can be used directly in Claude Code via the claude-api skill:

Run `/claude-api build-eval` if you want to generate an evaluation set for a particular problem. You can steer it by providing access to examples (e.g., traces). Claude will employ the guidance shared in this article to design the examples and grader, and ensure you approve the examples and the grader.

Run `/claude-api hillclimb` if you have an evaluation and want Claude to improve on this, guided by your goal (e.g., better performance, or lower cost while performance holds). Claude will employ the guidance shared in this article to check for overfitting while climbing and check for bugs in the eval itself, such as a grader that marks a correct-looking answer wrong or a harness error, both before the first round and whenever the score stalls.

With special thanks to Misha Khalman for skill development. With thanks to Misha Khalman, Michael Segner, Matt Bell, Matt Thanabalan, and Punit Shah for reviews, contributions, and product support.

Agentic Monitoring

BEN CONGDON · 23 OCT 2026 · SOURCE

A recent Anthropic blog post contained a short paragraph, not at all the point of the post, which has been buzzing around my brain since I read it:

> A new level of operational excellence is possible. Our team uses Claude Tag heavily, which has helped us monitor each component of the system. For example, we created Slack channels where we tasked Claude with being obsessively focused on the health of a single component. We call these Claudes "superfans." We have a scion-daemon superfan, a scion-cache superfan, an Antique merge queue superfan, and so on. They have access to our telemetry and logs, and act like an experienced engineer who has nothing better to do than to refresh charts and look for anything strange. We also have traditional alerts for all of our systems, but our superfans can detect subtle and novel issues that make these alerts seem one-dimensional by comparison.

This is *so obviously* the path forward for software systems operations. I've been thinking about monitoring with agents for the past few weeks, and this paragraph crystallized some of those thoughts.

Modern software systems, like the LLMs that write them, are becoming more "grown" than "designed". Yes, human engineers often still give the high-level design and shape to the system, and yes, software systems (for now) are much more interpretable than LLMs – since you can just sit down and read the code, if you are so inclined. However, with the volume of code being written and shipped increasing exponentially, the amount of actual eyes-on-code knowledge that any human operator of a system has is decreasing.

Systems monitoring was never *easy*. There is a whole industry of observability SaaS companies – Sentry, Datadog, Splunk, Honeycomb, to name a few. Observability is expensive.[1] Doing it well requires a specialized human skill set and relies on building experience operating real systems at scale. One can really only develop these skills by working in industry. This is a domain that takes years to train junior engineers on.

As with seemingly everything in the software industry in 2026, system monitoring is poised for a step change due to agents. I… am surprised I have not seen more people write about this, especially in the current milieu of "agent frenzy". It's right there in front of us all, and it *works*. There have been a bunch of startups trying to do the "AI SRE", but very little writing from practitioners.

But you don't really need a full "AI SRE" to improve operational excellence, nor do you need a built-out SaaS product to do so. As of the Opus 5.5 / Astra class of models, with a bit of care, it's entirely possible to make a good agent-driven monitoring system that "just works".

Case study: change-point monitoring

I'll ground this discussion in a problem that is near to my heart: production monitoring of change points, like binary releases and config pushes. The idea is that you want to do specialized, intensive monitoring around critical points of change to a system; these change points are more likely to trigger a regression, and thus if you monitor these events closely, you can limit its impact. To quote the clichéd line from the Google SRE book, "~70% of outages are due to changes in a live system".

This is a wicked problem. I've been working on it in some form since 2023, and it's been hard to get traction on. From talking with folks across the industry, I've learned there are ways to do change-point monitoring "well", but these methods largely still rely on humans to do the right thing (e.g. tagging metrics/alerts diligently, defining precise signals, etc.) and require significant organizational investment.

More specifically, achieving "takeoff" for change-point monitoring as a piece of platform engineering is hard because:

- Your first instinct may be to target highly motivated users who already have a good ops posture. However, these teams' rollouts are already mostly safe, and you don't actually raise the ops waterline much. The thing you're selling here is *toil reduction*, which is "nice to have" but is not itself a first-order safety improvement. Precision is easier to achieve for highly motivated teams, but this means the change-point system is "competing" on recall and latency with traditional alerts.

- You can alternatively target the broad population of services. In other words, you go super wide and opt in "all services", on a chosen subset of universal health metrics. In practice, most of these general health metrics are irreducibly noisy. This noise results in poor precision and alert fatigue.

- Prior to LLMs, there was a clear precision/latency tradeoff as well. Along this spectrum are: structured logs (precise, slow), metrics (coarse, fast), and alerts (even coarser, recall is pretty bad and precision is pretty bad, but "at least it's something").

Shockingly, agents largely "just" solve this problem. Agents are happy to run copious queries, can write per-change queries on the fly, and will scan logs ad nauseam. As a result, you can get both good precision and recall for cheap. In the basic version of this setup, you just need the ability to connect your codebase (e.g. to reason about config consumption) and your observability stack (e.g. to view and monitor the live production state) to the agent.

I've spiked a prototype of exactly this: an agent that proposes a monitoring plan based on the service code, a scaffold that executes that plan, and an agent that investigates triggered monitors, with an escalation path to humans. Even in this early prototype, I've been able to detect regressions in systems I have *zero* understanding of with precision and latency that would have been impossible a year ago.

In my iteration on this, I've found that a pure agent loop is not the optimal shape for agentic monitors. Rather, a harness that allows the agent to write a spec of how to monitor, along with a deterministic system for running those specs, produces significantly cleaner signals with dramatically lower token usage. To the extent there is "secret sauce" in this pattern, it's in the noise-reduction heuristics, the prompts that get the agent to write good specs, and a labeled eval set large enough to validate further tweaks.[2]

Agents are structurally well suited to monitoring

Two mindset shifts need to happen before this pattern can catch on:

1. Agents actually can do a much better job at monitoring than humans.

2. Monitoring agents need scaffolding, the same way the rest of the software development lifecycle had to adapt for coding agents. There hasn't been a "Claude Code moment" for monitoring yet.

Once you make these shifts, monitoring agents have several "unfair advantages" over human operators:

- Ability to track everything, 24/7. If tokens are cheap, then any regression in a metric you care about and any suspicious log line can be investigated – not just the events that would previously have risen above the "human attention cutline". Agents don't need to "follow the sun".

- Ability to fan out and investigate several paths toward resolving a production problem. Agents can parallelize investigations when multiple things go wrong at once, and coordinate quickly. This is already my go-to for incident triage: I just launch a bunch of agents to

investigate several potential threads or theorized incident causes. Right now, this is human-driven, but there's really no reason it couldn't be agent-initiated in the future. You can elastically scale up a fleet of incident responders.
    - We're already seeing that this is becoming critical for security incidents. Teams/fleets/swarms of agents are being used for offensive cyberattacks, and defenders are beginning to build up similar defensive capabilities (e.g. OpenAI's Daybreak Blue, Anthropic's Project Glasswing, Google's Fairwind).

- Ability to respond faster to a problem than can a human. Agents can read code faster than humans. Agents can also query and digest metrics and log signals significantly faster. This speed doesn't always translate into faster incident detection time; a human with a deep understanding of a system may still be faster at finding the right signal and intuiting the root causes. But to the extent that tacit knowledge can be recorded and made available to an agent, the agent will eventually beat the human on speed.
    - By extension, for systems where the human operator isn't deeply experienced, the "time to understanding" is going to be shorter for an agent.

"A new level of operational excellence is possible."

Be more diligent: You wouldn't pay a full-time software engineer to look through all the Git commits for a service, pick out things to monitor, and make special short-term fast-trigger alerts just on those changes. However, for a couple cents per million tokens, it's possible now, and will become an industry standard.

Continuously improve performance: Agents can monitor performance, look at heap dumps and stack traces and CPU profiles, and propose optimizations from the patterns they find. Put the proposed improvements behind A/B tests if necessary, and monitor as you would any other change to the service.

In one instance, I observed that a few sessions of Astra/Opus investigation discovered 50% heap savings and 30% latency reduction in a common hot-path library that humans had already spent nontrivial effort to optimize. Once you give a sufficiently powerful LLM a VM and benchmarks to run, the "hill-climbing" ability of current-generation LLMs is remarkable.

Take action: Thus far, I've mostly been discussing "read-only" monitoring systems. Agents, of course, needn't be "read-only". With the right harness and guardrails in place, there's definitely a future in which agents will be able to take front-line actions to remediate incidents. This could span anything from well-scoped operational tasks like resizing instances or rotating compute pools, to deterministic actions like rolling back a config change or binary, to open-ended tasks like root-causing bugs, proposing fixes, and redeploying on-the-fly. Obviously there is risk the

more control you hand-over to the agent, but considering how much agent capabilities expanded over the course of the past 12 months, the amount of write-path work that can be responsibly handed over to agents is bound to increase. The promise of truly self-healing systems is exciting.

Cost is no longer a limitation. Luna/Haiku-class models[3] are now competent graph-starers. A moderately large tech company could likely monitor its entire fleet for on the order of $100/day. Startup-scale fleets would be a small fraction of that. Compared to human labor, or even the cost of the infra that runs observability, this is trivial.

Create a virtuous cycle: If done well, agentic monitoring creates a positive feedback loop: coding agents allow you to ship more, monitoring agents allow you to support more systems, which in turn frees up human time to direct more things to ship. Humans increasingly spend time "building the factory" or "tending the garden".

Agentic monitoring is worth investing in. It will soon be reckless not to have agents monitoring your systems. Until the rest of the industry catches up, getting started now affords an unfair advantage.

1. A set of optimizations my team recently completed saved us several million dollars in annualized observability cost, and… that was a mere fraction of what *just my team* spends on observability. ↩

2. Much, much more could be written about the design of agentic monitoring harnesses. This is one of the clearest "we're so early" feelings I've had in a while. General harnesses, like Anthropic's "Claude Tag", evidently already do a good job at agentic monitoring, per the original post. However, based on what I've seen, even a little monitoring-specific harness engineering picks up a lot of low-hanging fruit. ↩

3. Here I mean GPT-6-Luna and Claude Haiku 5.5. ↩

What requirements don't you have?

SEANGOEDECKE.COM RSS FEED · 19 OCT 2026 · SOURCE

When designing software, the requirements you don't have are as important as the requirements you do. Identifying what you can compromise on — what can be slow, or stale, or inflexible — is usually the key to an elegant design.

Elegant systems drop some requirements

[SVG OMITTED]

For instance, take preforking web servers like Unicorn. Because they lack the feature of shared mutable state between server instances, they can run each server in its own process, which provides isolation for free. Because Unicorn specifically ignores the problem of slow client handling, it can queue requests directly on Unix sockets, drastically simplifying the implementation.

Because S3 does not support atomic multi-key updates — you cannot update multiple blobs in a single transaction — it can store data at the blob level however it chooses. That makes it much faster, since reads and writes don't have to cross storage partitions, and much easier to scale, since less coordination means fewer sources of truth.

StatsD uses UDP by default, which means it doesn't guarantee that every piece of data you send makes it into storage. In exchange for dropping that requirement, it incurs much less network overhead, which is a big advantage for a stats aggregation service that's constantly shoveling data over the wire.

The general design approach of fanning out data to an edge cache buys a lot of speed and scaling by dropping the requirement that data must always be up-to-date. SQLite buys operational simplicity — it can run out of a single file! — by dropping the requirement of concurrent writes. By accepting high tail latency, Cursor can scale their distributed git backend more easily. Well-designed systems turn their weaknesses into strengths.

Why dropping requirements is hard

[SVG OMITTED]

Anyone who's spent time designing real-world software (or who's studied distributed systems) knows that system design is a process of compromise. If you attempt to satisfy every single potential requirement — speed, concurrency, up-to-date data, a simple interface, supporting complex user behaviors — you will fail to design a working system at all.

Figuring out what requirements you can relax is difficult because **it combines both business and technical expertise**. It is technically challenging to identify precisely the level of staleness that a particular design change will allow; it is likewise challenging from a business perspective to determine whether that level of staleness is acceptable to users.

In most cases, an engineer who understands the technical side must try and explain it to a product manager, who can make a judgement on the business side. But this is a very difficult partnership to get right. Engineers and PMs have different incentives and little ability to check each other's work, which makes it hard to build trust. Without trust, the product manager is liable to avoid giving any ground at all, which leaves the design in an impossible state.

This is an underappreciated reason why solo developers or startups can sometimes run rings around larger companies. Because the technical and business decisions are united in a single person, they can make design tradeoffs that would be impossible for a large company, enabling them to build systems that are far more elegant and powerful.

Working with LLMs

[SVG OMITTED]

Knowing what requirements you don't have becomes far more important when working with coding agents. Frontier models are very capable, but they have all gone through the purifying flames of RLVR: like high-achieving schoolchildren, they are obsessed with satisfying the Grader. On top of that, they have very little business context compared to an experienced human. That often manifests as them catching correctness or consistency "bugs" that are really just hypothetical unsatisfied requirements.

For instance, it's not uncommon for an LLM to casually introduce a mutex or other concurrency mechanism to ensure that two pieces of semi-related data are kept in sync, even if not told to do so. I've seen LLMs add weird checks in the middle of request handling to ensure that some already-checked piece of data hasn't changed mid-flight, even if that wouldn't successfully prevent the feared TOCTOU bug. LLMs are obsessed with edge cases like "what if the current user loses their admin status in the middle of us handling one of their admin requests?", or "what if the cache in storage partition A is updated a nanosecond before the one in storage partition B, leading some users to see different data?". Particularly if run in a loop or told to "make no mistakes", they will pile on cases like these until they've transformed your four-line change into a thousand-line monster.

This is why I'm confused by some people's claims that code review is dead. About half the time I review a large AI-generated change, I notice some unnecessary requirement that makes the diff more complex. When I dig deeper, the LLM invariably introduced this requirement on its own.

When working with a cache, I will often spell out in my prompt that I don't mind the data sometimes being stale or briefly inconsistent between cache instances. When reviewing Golang code, the first thing I do is scan for added mutexes or layers of indirection that don't look necessary to me.

If you allow requirements like these to propagate unchecked through a codebase, they will petrify the system over time. The whole thing will be such a mess of checking and re-checking and holding locks and enforcing consistency that it cannot be scaled at all. Keeping systems elegant requires a ruthless focus: on accepting stale data or occasional tail latency, on avoiding all but critical instances of coordination, and in general on figuring out what requirements you *don't* have to satisfy.